

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

-----**-----



BÁO CÁO MÔN HỌC

Học phần: IT6130 – Lập trình và ảo hóa cho IoT

Đề tài 2.4

VIRTUAL SMART AIR QUALITY GATEWAY

Hệ thống Virtual IoT Gateway giám sát chất lượng không khí đô thị

Giảng viên hướng dẫn: **TS. Đặng Tuấn Linh – TS. Trần Hải Anh**

Nhóm sinh viên thực hiện

STT	Họ và tên	MSSV
1	Nguyễn Trường Chinh	20261101M
2	Nguyễn Hà Minh	20261102M
3	Đỗ Thành Đạt	20261103M

Hà Nội, năm 2026

MỤC LỤC

MỤC LỤC	2
PHÂN CÔNG NHIỆM VỤ CHI TIẾT	5
Thành viên 1 – Nguyễn Trường Chinh (20261101M)	5
Thành viên 2 – Nguyễn Hà Minh (20261102M)	5
Thành viên 3 – Đỗ Thành Đạt (20261103M)	6
MỞ ĐẦU	8
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	9
1.1. Bối cảnh và động lực	9
1.2. Mục tiêu của dự án	9
1.3. Phạm vi bài toán	9
1.4. Công nghệ sử dụng	10
CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC HỆ THỐNG.....	11
2.1. Kiến trúc tổng quan	11
2.2. Các thành phần dịch vụ	11
2.3. Thiết kế cây topic MQTT	11
2.4. Thiết kế định dạng message (JSON)	12
2.4.1. Telemetry (sensor → gateway)	12
2.4.2. Command (gateway → actuator)	12
2.4.3. Status (actuator → gateway/API).....	12
2.4.4. Event (gateway sinh ra).....	12
CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT CHI TIẾT	13
3.1. Virtual Sensor	13
3.2. Virtual Actuator	13
3.3. Edge Gateway – Chuẩn hóa và điều phối.....	13
3.4. Tính chỉ số AQI	14
3.5. Rule engine	14
3.6. Lưu trữ InfluxDB.....	15
3.7. Tích hợp ThingsBoard (uplink + RPC)	15
3.8. REST API (FastAPI)	15
3.9. Control Web dashboard	16
3.10. Grafana dashboard	16
3.11. Docker Compose và triển khai	16

CHƯƠNG 4. PHÂN CÔNG NHIỆM VỤ VÀ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN.....	17
4.1. Nguyên tắc phân công	17
4.2. Nguyễn Trường Chinh (20261101M) – Thiết bị ảo & Thiết kế giao tiếp MQTT.....	17
a) Thiết kế cây topic MQTT và định dạng message JSON.....	17
b) Cài đặt Virtual Sensor (sensor.py)	17
c) Cài đặt Virtual Actuator (actuator.py)	18
d) Cấu hình broker và đóng gói.....	18
4.3. Nguyễn Hà Minh (20261102M) – Edge Gateway & Xử lý dữ liệu	18
a) Lỗi điều phối Gateway (gateway.py)	18
b) Tính chỉ số AQI (aqi.py).....	19
c) Rule engine (rules.py)	19
d) Lưu trữ InfluxDB (influx.py) và đóng gói	19
4.4. Đỗ Thành Đạt (20261103M) – Cloud, REST API, Dashboard & Triển khai	19
a) Tích hợp ThingsBoard Cloud (thingsboard.py + RPC handler)	20
b) REST API (api/main.py).....	20
c) Control Web dashboard (control-web).....	20
d) Trực quan hóa và triển khai	20
4.5. Bảng tổng hợp đóng góp.....	21
4.6. Công việc thực hiện chung	21
CHƯƠNG 5. GIẢI ĐÁP CÁC CÂU HỎI BẮT BUỘC	22
Câu 1. Vì sao nên tính AQI tại gateway thay vì gửi dữ liệu thô lên cloud rồi mới tính?.....	22
Câu 2. Nhóm thiết kế topic và message format cho quan trắc không khí thế nào?	22
Câu 3. Cách tính AQI và xác định chất ô nhiễm dominant của nhóm?	22
Câu 4. Rule engine điều khiển lọc khí/quạt theo cấp AQI ra sao?.....	22
Câu 5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?	22
Câu 6. So sánh dashboard Grafana (edge) và bản đồ AQI trên ThingsBoard (cloud)?	23
Câu 7. Vì sao container không nên dùng localhost để gọi service khác?	23
Câu 8. Muốn mở rộng thành mạng quan trắc toàn thành phố cần thay đổi gì?.....	23
CHƯƠNG 6. KẾT QUẢ VÀ ĐÁNH GIÁ	24
6.1. Kết quả đạt được.....	24
6.2. Đối chiếu tiêu chí chấm điểm	24
6.3. Một số yêu cầu nâng cao đã thực hiện.....	24
6.4. Hạn chế và hướng phát triển.....	25
KẾT LUẬN.....	26

PHÂN CÔNG NHIỆM VỤ CHI TIẾT

Thành viên 1 – Nguyễn Trường Chinh (20261101M)

Mảng phụ trách: Thiết bị ảo (sensor, actuator) và thiết kế lớp giao tiếp MQTT.

Hạng mục công việc	Công việc chi tiết	File / Sản phẩm
Thiết kế cây topic MQTT	Thiết kế cây topic phân cấp theo station_id gồm 5 nhánh: sensor/telemetry, actuator/command, actuator/status, gateway/normalized, gateway/event; quy ước dùng QoS 1 cho mọi bản tin và cờ retain cho status của actuator; đảm bảo thêm trạm mới chỉ cần đổi station_id.	(thiết kế)
Thiết kế message JSON	Định nghĩa schema JSON cho telemetry, command và status; thống nhất các trường định danh device_id, station_id và timestamp ISO-8601 UTC; thống nhất tên trường nồng độ (pm2_5, pm10, co_ppm, no2_ppb, o3_ppb) dùng chung toàn hệ thống.	(thiết kế)
Virtual Sensor	Đọc cấu hình từ env; bảng baseline theo từng trạm; hàm diurnal_factor mô phỏng 2 đỉnh cao điểm 8h/18h; random-walk step_toward tạo xu hướng mượt; sinh tình huống đột biến (PM $\times$ 3,2 và CO $>$ 9 ppm) mỗi ~40 vòng; O ₃ ngược pha giao thông; publish JSON QoS 1 theo chu kỳ; reconnect tự động; tắt sạch bằng SIGTERM/SIGINT.	sensor.py
Virtual Actuator	Subscribe topic command (QoS 1); máy trạng thái cho fan (off/on/max), purifier, mist, board; hàm apply_command phân tích {target, action, reason}; publish_status với retain=True; công bố trạng thái ban đầu khi kết nối; reconnect tự động.	actuator.py
Broker & đóng gói	Cấu hình Mosquitto (listener 1883, allow_anonymous, persistence, logging); viết Dockerfile cho sensor và actuator; khai báo tham số môi trường 3 cặp sensor-actuator.	mosquitto.conf, Dockerfile

Thành viên 2 – Nguyễn Hà Minh (20261102M)

Mảng phụ trách: Edge gateway – chuẩn hóa, tính AQI, rule engine và lưu trữ dữ liệu.

Hạng mục công việc	Công việc chi tiết	File / Sản phẩm
Lỗi điều phối Gateway	Cài đặt MQTT client của gateway: subscribe đồng thời 2 wildcard city+/sensor/telemetry và city+/actuator/status (QoS 1); định tuyến bản tin theo hậu tố topic; thiết kế bộ	gateway.py

Hạng mục công việc	Công việc chi tiết	File / Sản phẩm
	nhớ LATEST lưu bản đo, AQI và thời điểm cập nhật mới nhất từng trạm.	
Chuẩn hóa dữ liệu	Hàm validate_normalize: kiểm tra các trường bắt buộc (station_id, pm2_5, pm10) và loại bỏ bản ghi thiếu; ép kiểu float cho các trường số; kẹp giá trị âm về 0 để chống dữ liệu lỗi.	gateway.py
Điều phối pipeline xử lý	Hàm handle_telemetry điều phối: tính AQI → cập nhật LATEST → ghi InfluxDB → publish normalized → chạy rule engine → publish command và event → đẩy ThingsBoard; hàm handle_status cập nhật trạng thái actuator; log chi tiết từng bước.	gateway.py
Tính chỉ số AQI	Số hóa bảng breakpoint US EPA cho 5 chất; hàm aqi_for nội suy tuyến tính AQI thành phần, kẹp trần khi vượt bảng; hàm compute_aqi lấy AQI tổng = max thành phần và xác định dominant pollutant; hàm category và board_label phân cấp nhãn.	aqi.py
Rule engine	Bảng ngưỡng THRESHOLDS tách rời logic (chỉnh động qua Shared Attributes); 4 luật theo cấp AQI: AQI>150 (purifier+fan+board Unhealthy), AQI>200 (thêm mist), CO>9 ppm (fan max), AQI≤50 (tắt hết); sinh song song command và event, gán chuỗi action_taken.	rules.py
Lưu InfluxDB & đóng gói	Module ghi InfluxDB v2 dạng thủ tục (init + write_api SYNCHRONOUS, bắt lỗi ghi); thiết kế 3 measurement air_quality, gateway_events, actuator_status với tag/field đúng; viết Dockerfile cho gateway.	influx.py, Dockerfile

Thành viên 3 – Đỗ Thành Đạt (20261103M)

Mảng phụ trách: Tích hợp Thingsboard Cloud với RPC, REST API, dashboard Thingsboard + Grafana và triển khai toàn hệ thống.

Hạng mục công việc	Công việc chi tiết	File / Sản phẩm
Tích hợp ThingsBoard – uplink	Hàm send_telemetry đóng gói telemetry nhiều trạm kèm AQI theo định dạng {station: [{ts, values}]} và publish lên v1/gateway/telemetry; hàm connect_device báo thiết bị con online lên v1/gateway/connect.	thingsboard.py

Hạng mục công việc	Công việc chi tiết	File / Sản phẩm
Tích hợp ThingsBoard – RPC	Subscribe v1/gateway/rpc; phân tích {device, method, params}; gọi callback xử lý và publish reply; cơ chế bật/tắt qua TB_ENABLED (no-op khi false) để hệ thống local chạy độc lập.	thingsboard.py
RPC handler tại gateway	Hàm make_rpc_handler ánh xạ setFan/setPurifier/setMist/setBoard thành command MQTT gửi tới actuator và trả kết quả.	gateway.py
REST API	Ứng dụng FastAPI với MQTT background client (lifespan); subscribe normalized/event/status; lưu STATE và EVENTS (deque 50 event/trạm); 5 endpoint (/health, /stations, /stations/{id}/state, /stations/{id}/events, POST command); Pydantic model Command; xử lý lỗi 404/503.	api/main.py
Control Web dashboard	FastAPI proxy đứng trước air-api; phục vụ trang HTML (tiêm REFRESH_MS, STATIONS); giao diện mỗi trạm là card với AQI đổi màu theo cấp, nhãn category/dominant, lưới thông số, công tắc bật–tắt actuator gửi command, danh sách event và chỉ báo kết nối; tự làm mới theo chu kỳ.	control-web/
Trực quan hóa & triển khai	Dashboard Thingsboard Cloud phục vụ giám sát và điều khiển actuator. Provisioning Grafana (datasource InfluxDB Flux + dashboard); viết docker-compose.yml điều phối 12 service (mạng air-net, volume, healthcheck + depends_on service_healthy, ánh xạ cổng, biến môi trường); viết .env và README. Cấu hình cảnh báo Thingsboard Cloud qua Alarms.	docker-compose.yml, infra/, README

Công việc chung: Thống nhất kiến trúc và hợp đồng dữ liệu ở giai đoạn đầu, kiểm thử tích hợp end-to-end, dò – sửa lỗi liên service, chuẩn bị kịch bản demo và cùng biên soạn báo cáo, slide trình bày.

MỞ ĐẦU

Ô nhiễm không khí đô thị, đặc biệt là bụi mịn PM2.5, PM10 và các khí độc như CO, NO₂, O₃, đang là vấn đề nhức nhối tại các thành phố lớn. Nồng độ các chất này thay đổi liên tục theo thời gian trong ngày (cao điểm giao thông sáng và chiều) và theo từng khu vực, việc giám sát thủ công hoặc chỉ dựa vào một điểm đo là không đủ. Một hệ thống giám sát cần thu thập dữ liệu từ nhiều trạm, tính toán chỉ số chất lượng không khí (AQI) một cách kịp thời và tự động phản ứng khi mức ô nhiễm vượt ngưỡng nguy hiểm.

Trong khuôn khổ học phần IT6130 – Lập trình và ảo hóa cho IoT, nhóm thực hiện Đề tài 4: xây dựng một hệ thống Virtual Smart Air Quality Gateway hoàn chỉnh theo mô hình IoT thực tế, nhưng toàn bộ thiết bị đều được ảo hóa và triển khai bằng Docker Compose. Hệ thống mô phỏng nhiều trạm cảm biến gửi dữ liệu qua giao thức MQTT; một edge gateway đứng ở biên (edge) làm nhiệm vụ chuẩn hóa dữ liệu, tính AQI, chạy rule engine để tự động điều khiển các thiết bị chấp hành (quạt thông gió, máy lọc khí, phun sương, bảng cảnh báo); dữ liệu được lưu vào InfluxDB, trực quan hóa trên Grafana, đồng bộ hai chiều với ThingsBoard Cloud (telemetry và RPC) và cung cấp REST API cùng một Web dashboard để truy vấn, điều khiển thủ công.

Phương pháp thực hiện là ảo hóa toàn bộ hệ thống: mỗi thành phần (sensor, actuator, broker, gateway, database, dashboard, API) là một service độc lập chạy trong container Docker, giao tiếp với nhau qua mạng nội bộ theo tên service. Cách làm này giúp nhóm tái hiện đầy đủ kiến trúc một hệ thống IoT quan trắc không khí mà không cần phần cứng vật lý, đồng thời rèn luyện kỹ năng lập trình mạng, xử lý dữ liệu thời gian thực, tính toán chỉ số môi trường và triển khai microservice bằng container.

Kết quả cho thấy hệ thống đáp ứng đầy đủ các yêu cầu bắt buộc và một số yêu cầu nâng cao của đề tài.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và động lực

Chất lượng không khí là chỉ số môi trường có tác động trực tiếp đến sức khỏe cộng đồng. Các chất ô nhiễm chính gồm bụi mịn PM2.5, bụi PM10 và các khí CO, NO₂, O₃ biến thiên mạnh theo giờ trong ngày và theo vị trí trạm đo. Bài toán đặt ra là phải phát hiện sớm các mức ô nhiễm nguy hiểm và đưa ra phản ứng kịp thời.

Nếu gửi toàn bộ dữ liệu thô lên cloud rồi mới xử lý, hệ thống sẽ phụ thuộc hoàn toàn vào đường truyền và độ trễ của cloud; khi mất kết nối thì việc cảnh báo và điều khiển bị gián đoạn. Vì vậy, nhóm lựa chọn mô hình edge computing: đặt một gateway ở biên để nhận dữ liệu từ nhiều trạm, chuẩn hóa, tính AQI, phát hiện sự kiện và điều khiển thiết bị ngay tại chỗ, đồng thời vẫn đồng bộ lên cloud để giám sát tập trung.

1.2. Mục tiêu của dự án

Nhóm đặt ra các mục tiêu cụ thể, bám sát yêu cầu của đề bài:

1. Lập trình nhiều virtual sensor mô phỏng các trạm quan trắc không khí khác nhau, dữ liệu có xu hướng theo thời gian chứ không random độc lập.
2. Xây dựng edge gateway nhận telemetry qua MQTT từ tất cả các trạm.
3. Thiết kế cây topic MQTT phân cấp và định dạng message JSON cho quan trắc không khí, có khả năng mở rộng.
4. Chuẩn hóa dữ liệu và tính chính xác chỉ số AQI, xác định chất ô nhiễm dominant.
5. Xây dựng rule engine tự động điều khiển quạt / máy lọc khí / phun sương / bảng cảnh báo theo cấp AQI.
6. Lưu trữ telemetry, AQI, event và trạng thái actuator vào InfluxDB; trực quan hóa trên Grafana.
7. Tích hợp ThingsBoard Cloud theo Gateway API: đẩy telemetry nhiều trạm kèm AQI và nhận lệnh điều khiển từ xa (RPC).
8. Cung cấp REST API (FastAPI) và một Web dashboard để truy vấn trạng thái và điều khiển thủ công.
9. Đóng gói và triển khai toàn bộ hệ thống bằng Docker Compose.

1.3. Phạm vi bài toán

Hệ thống mô phỏng một mạng quan trắc gồm ba trạm station-01, station-02, station-03. Mỗi trạm có một virtual sensor node (đo PM2.5, PM10, CO, NO₂, O₃, nhiệt độ, độ ẩm) và một virtual actuator node (quạt thông gió fan, máy lọc khí purifier, phun sương mist, bảng cảnh báo board). Mỗi trạm có mức nền ô nhiễm khác nhau để dashboard phân biệt được. Toàn bộ thiết bị được ảo hóa, không sử dụng phần cứng vật lý.

1.4. Công nghệ sử dụng

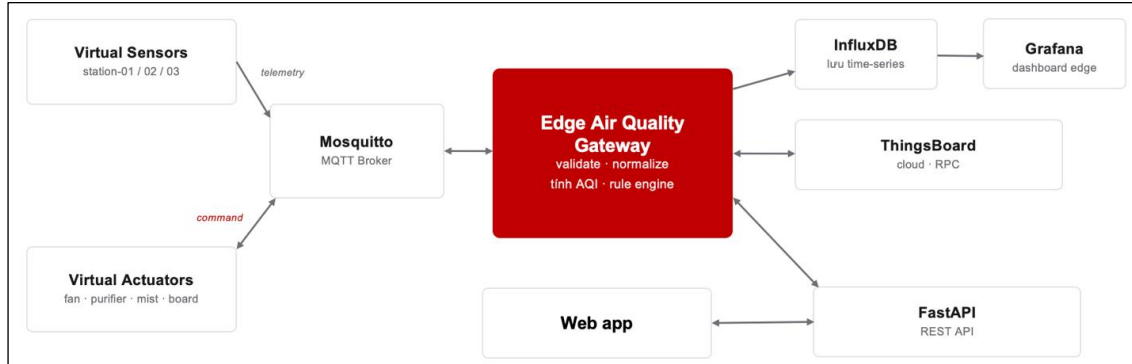
Thành phần	Công nghệ	Vai trò
Ngôn ngữ	Python 3.10+	Toàn bộ sensor, actuator, gateway, API
Giao thức	MQTT (paho-mqtt)	Truyền telemetry, command, status, event
Message broker	Eclipse Mosquitto 2	Trung chuyển bản tin giữa các thiết bị
CSDL time-series	InfluxDB 2.7	Lưu telemetry, AQI, event, trạng thái
Dashboard edge	Grafana 11.3	Trực quan hóa dữ liệu quan trắc
REST API / Web	FastAPI + Uvicorn	Truy vấn và điều khiển thủ công
Cloud IoT	ThingsBoard Cloud	Telemetry tập trung + điều khiển RPC
Triển khai	Docker + Docker Compose	Đóng gói và điều phối 12 container

Bảng 1.1. Các công nghệ sử dụng trong hệ thống

CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC HỆ THỐNG

2.1. Kiến trúc tổng quan

Hệ thống được tổ chức theo mô hình edge–cloud. Các virtual sensor publish telemetry lên MQTT broker; edge gateway subscribe toàn bộ, xử lý (validate, normalize, tính AQI, chạy rule engine), ghi InfluxDB, publish dữ liệu chuẩn hóa và command điều khiển, đồng thời đồng bộ hai chiều với ThingsBoard Cloud. REST API và Web dashboard đứng song song để truy vấn trạng thái và điều khiển thủ công.



Hình 2.1. Kiến trúc tổng quan hệ thống Virtual Smart Air Quality Gateway

2.2. Các thành phần dịch vụ

Toàn bộ hệ thống gồm 12 container cùng nằm trên một mạng bridge nội bộ air-net và gọi nhau bằng tên service (DNS nội bộ của Docker), tuyệt đối không dùng localhost.

Service	Số lượng	Mô tả chức năng
mosquitto	1	MQTT broker trung tâm (cổng 1883)
sensor-station-0X	3	Virtual sensor mỗi trạm, publish telemetry
actuator-station-0X	3	Virtual actuator mỗi trạm, nhận command
air-gateway	1	Edge gateway: AQI, rule engine, TB sync
influxdb	1	CSDL time-series (cổng 8086)
grafana	1	Dashboard edge (cổng 3000)
air-api	1	REST API FastAPI (cổng 8000)
control-web	1	Web dashboard điều khiển (cổng 8080)

Bảng 2.1. Danh sách các service trong hệ thống

2.3. Thiết kế cây topic MQTT

Nhóm thiết kế cây topic phân cấp theo station_id để mỗi trạm có không gian tên riêng, tách rõ từng luồng dữ liệu. Vì vậy, thêm một trạm mới chỉ cần đặt station_id mới không phải sửa logic gateway.

city/{station_id}/sensor/telemetry	# dữ liệu đo thô từ sensor
city/{station_id}/actuator/command	# lệnh tới fan / purifier / mist / board
city/{station_id}/actuator/status	# phản hồi trạng thái actuator (retained)

```
city/{station_id}/gateway/normalized    # dữ liệu chuẩn hóa + AQI đã tính
city/{station_id}/gateway/event         # event cảnh báo do gateway sinh ra
```

Gateway subscribe theo wildcard city/+sensor/telemetry và city/+actuator/status để nhận dữ liệu của mọi trạm mà không cần cấu hình cứng danh sách trạm.

2.4. Thiết kế định dạng message (JSON)

Mọi bản tin dùng JSON, thống nhất các trường định danh device_id, station_id và timestamp theo chuẩn ISO-8601 UTC. Bốn loại message chính:

2.4.1. Telemetry (sensor → gateway)

```
{
  "device_id": "sensor-station-01", "city_id": "hanoi",
  "station_id": "station-01",
  "pm2_5": 165.0, "pm10": 210.0, "co_ppm": 4.5,
  "no2_ppb": 80.0, "o3_ppb": 60.0,
  "temperature": 31.2, "humidity": 70.0,
  "timestamp": "2026-06-18T08:00:05Z"
}
```

2.4.2. Command (gateway → actuator)

```
{
  "station_id": "station-01", "target": "purifier",
  "action": "on", "reason": "aqi_unhealthy",
  "timestamp": "2026-06-18T08:00:05Z"
}
```

2.4.3. Status (actuator → gateway/API)

```
{
  "device_id": "actuator-station-01", "station_id": "station-01",
  "fan": "on", "purifier": "on", "mist": "off", "board": "Unhealthy",
  "last_command_reason": "aqi_unhealthy",
  "timestamp": "2026-06-18T08:00:06Z"
}
```

2.4.4. Event (gateway sinh ra)

```
{
  "station_id": "station-01", "event_type": "aqi_unhealthy",
  "severity": "warning", "aqi": 168, "dominant": "pm2_5",
  "value": 165.0, "threshold": 150,
  "action_taken": "purifier_on,fan_on,board_Unhealthy",
  "timestamp": "2026-06-18T08:00:06Z"
}
```

CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT CHI TIẾT

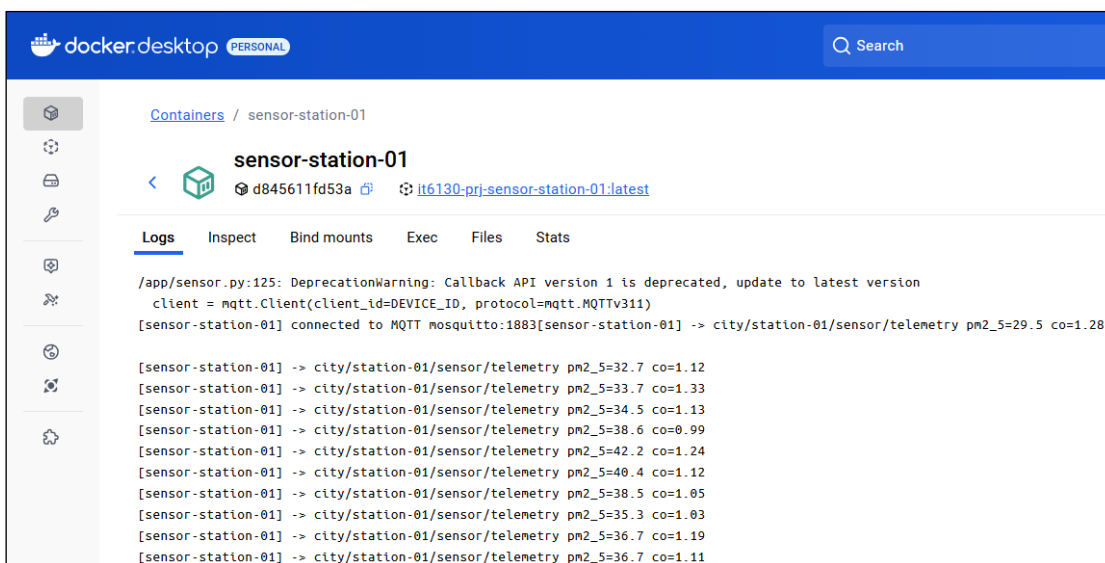
Chương này mô tả chi tiết thiết kế và cài đặt của từng service. Mỗi service là một chương trình Python độc lập, đóng gói bằng Dockerfile riêng và cấu hình hoàn toàn qua biến môi trường.

3.1. Virtual Sensor

Sensor mô phỏng dữ liệu quan trắc có ý nghĩa: nồng độ các chất biến thiên theo nhịp ngày và có xu hướng, không phải các số ngẫu nhiên rời rạc. Các cơ chế mô phỏng gồm:

- **Baseline theo trạm:** mỗi trạm có một mức nền riêng (STATION_BASE) cho PM2.5, PM10, CO, NO₂, O₃; ví dụ station-03 ô nhiễm nặng hơn station-02, giúp dashboard phân biệt được các trạm.
- **Nhịp ngày (diurnal):** hàm diurnal_factor() dùng hai đường Gaussian đặt tại 8h và 18h để tái hiện hai đỉnh cao điểm giao thông sáng và chiều.
- **Xu hướng mượt (random walk):** hàm step_toward() di chuyển giá trị hiện tại dần về giá trị mục tiêu cộng nhiễu nhỏ, tạo đường cong liên tục thay vì nhảy ngẫu nhiên.
- **Tình huống đột biến:** cứ khoảng 40 vòng, sensor kích hoạt một đợt ô nhiễm kéo dài 4–8 vòng, nhân PM lên 3,2 lần và đẩy CO vượt ngưỡng 9 ppm để kích rule engine phản ứng.
- **O₃ ngược pha giao thông:** ozone được mô phỏng biến thiên ngược với cao điểm giao thông cho sát thực tế.

Sensor đọc cấu hình từ biến môi trường (CITY_ID, STATION_ID, DEVICE_ID, MQTT_BROKER, MQTT_PORT, PUBLISH_INTERVAL), publish JSON telemetry theo chu kỳ với QoS 1, có cơ chế reconnect tự động (reconnect_delay_set 1–30s) và thoát sạch khi nhận SIGTERM/SIGINT.



The screenshot shows the Docker Desktop interface with the 'sensor-station-01' container selected. The 'Logs' tab is active, displaying the following output:

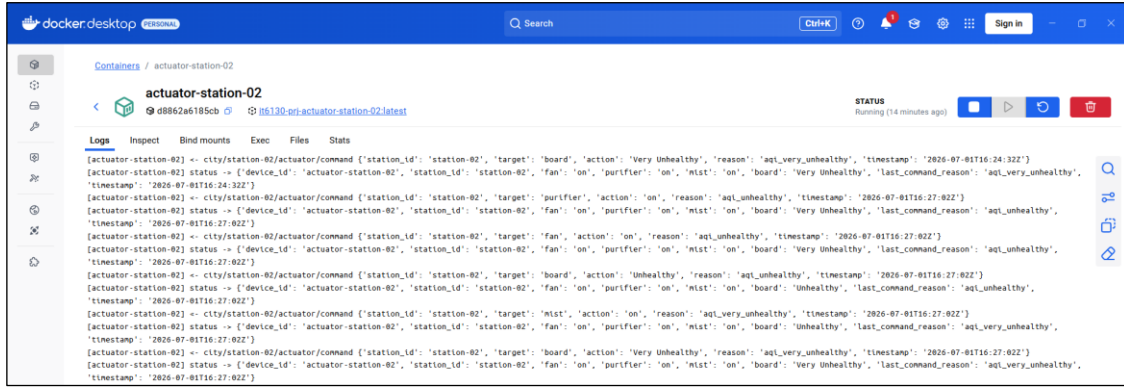
```

/app/sensor.py:125: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id=DEVICE_ID, protocol=mqtt.MQTTv311)
[sensor-station-01] connected to MQTT mosquitto:1883[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=29.5 co=1.28

[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=32.7 co=1.12
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=33.7 co=1.33
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=34.5 co=1.13
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=38.6 co=0.99
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=42.2 co=1.24
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=48.4 co=1.12
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=38.5 co=1.05
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=35.3 co=1.03
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=36.7 co=1.19
[sensor-station-01] -> city/station-01/sensor/telemetry pm2_5=36.7 co=1.11
  
```

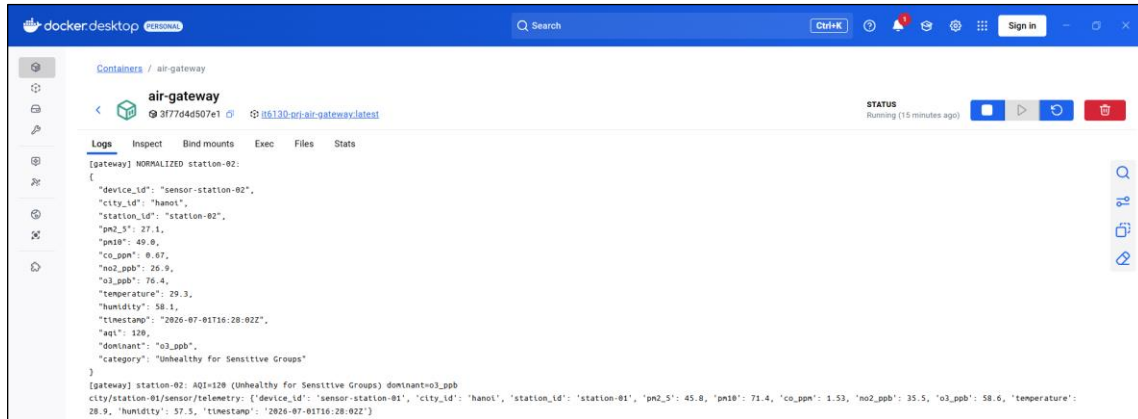
3.2. Virtual Actuator

Actuator là node chấp hành, quản lý bốn thiết bị: quạt thông gió (fan: off/on/max), máy lọc khí (purifier: off/on), phun sương giảm bụi (mist: off/on) và bảng cảnh báo (board: Good/Moderate/Unhealthy/Very Unhealthy). Luồng hoạt động: subscribe topic command → nhận lệnh từ gateway → cập nhật máy trạng thái STATE → publish status phản hồi.



Hàm `apply_command()` nhận một command JSON {target, action, reason}: nếu target là board thì action chính là nhãn hiển thị; nếu là fan/purifier/mist thì action là on/off/max. Sau mỗi lần đổi trạng thái, actuator gọi `publish_status()` với cờ `retain=True` để trạng thái mới nhất luôn tồn tại trên broker.

3.3. Edge Gateway – Chuẩn hóa và điều phối



Gateway là trung tâm xử lý. Module `gateway.py` subscribe đồng thời telemetry và status của mọi trạm, rồi điều phối toàn bộ pipeline xử lý:

10. Validate & normalize: hàm `validate_normalize()` kiểm tra các trường bắt buộc (station_id, pm2_5, pm10); nếu thiếu thì bỏ qua bản ghi. Các trường số được ép kiểu float và kẹp giá trị âm về 0.
11. Tính AQI cho bản đo (gọi module `aqi`).
12. Lưu trạng thái mới nhất từng trạm vào bộ nhớ LATEST (phục vụ REST API gián tiếp).
13. Ghi telemetry + AQI vào InfluxDB.

14. Publish dữ liệu normalized (telemetry kèm aqi, dominant, category) lên topic gateway/normalized.
15. Chạy rule engine, publish command tới actuator và event lên topic gateway/event, đồng thời ghi event vào InfluxDB.
16. Đẩy telemetry (kèm AQI) và trạng thái actuator lên ThingsBoard.

3.4. Tính chỉ số AQI

Module aqi.py tính AQI theo bảng breakpoint của US EPA bằng phương pháp nội suy tuyến tính. Với mỗi chất, tra khoảng nồng độ chứa giá trị đo và nội suy theo công thức:

$$I = (I_{hi} - I_{lo}) / (C_{hi} - C_{lo}) \times (C - C_{lo}) + I_{lo}$$

Trong đó C là nồng độ đo được; (C_lo, C_hi) là khoảng nồng độ và (I_lo, I_hi) là khoảng chỉ số AQI tương ứng. AQI tổng của trạm bằng giá trị lớn nhất trong các AQI thành phần; chất cho giá trị lớn nhất được xác định là dominant pollutant. Hàm category() phân AQI thành các nhãn Good / Moderate / Unhealthy for Sensitive Groups / Unhealthy / Very Unhealthy / Hazardous. Ví dụ PM2.5 = 165 µg/m³ cho AQI ≈ 168, lớn nhất trong các thành phần, nên dominant = pm2_5.

Cấp AQI	Nhãn phân loại
0 – 50	Good
51 – 100	Moderate
101 – 150	Unhealthy for Sensitive Groups
151 – 200	Unhealthy
201 – 300	Very Unhealthy
> 300	Hazardous

Bảng 3.1. Phân cấp AQI theo US EPA

3.5. Rule engine

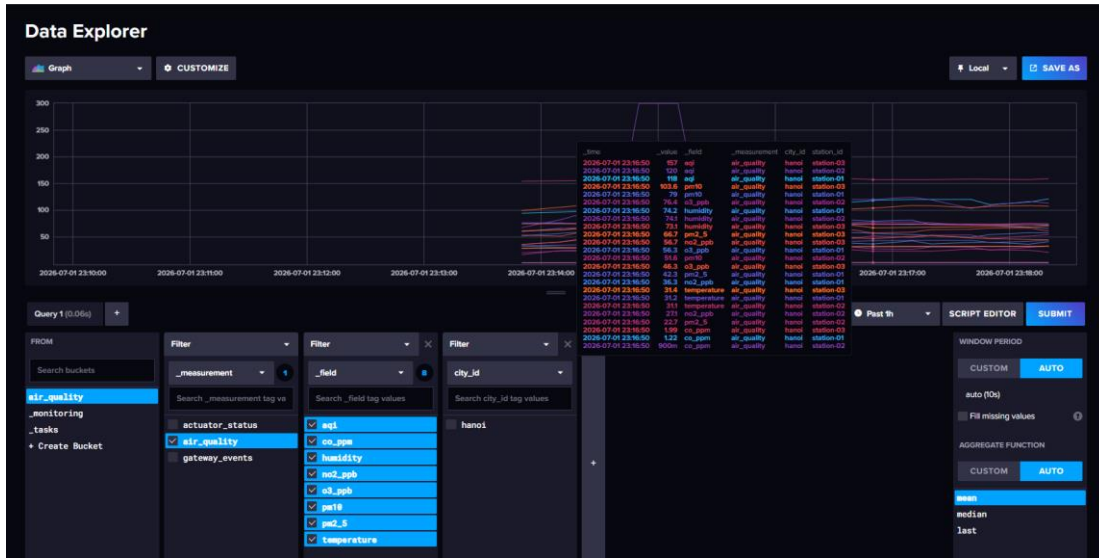
Module rules.py nhận nồng độ và kết quả AQI, sinh ra danh sách command (điều khiển actuator) và event (cảnh báo). Nhóm cài đặt bốn luật theo cấp AQI; các ngưỡng để trong THRESHOLDS nên có thể chỉnh từ bên ngoài (Shared Attributes của ThingsBoard):

Điều kiện	Hành động	Mức độ
AQI > 150 (Unhealthy)	Bật purifier + fan, board = Unhealthy	warning
AQI > 200 (Very Unhealthy)	Bật thêm mist, board = Very Unhealthy	critical
CO > 9 ppm	Bật fan mức max, sinh event co_high	critical
AQI ≤ 50 (Good)	Tắt fan/purifier/mist, board = Good	info

Bảng 3.2. Bộ luật điều khiển tự động theo cấp AQI

Mỗi event được gắn thêm chuỗi action_taken (liệt kê các hành động đã thực hiện) để phục vụ ghi log và dashboard.

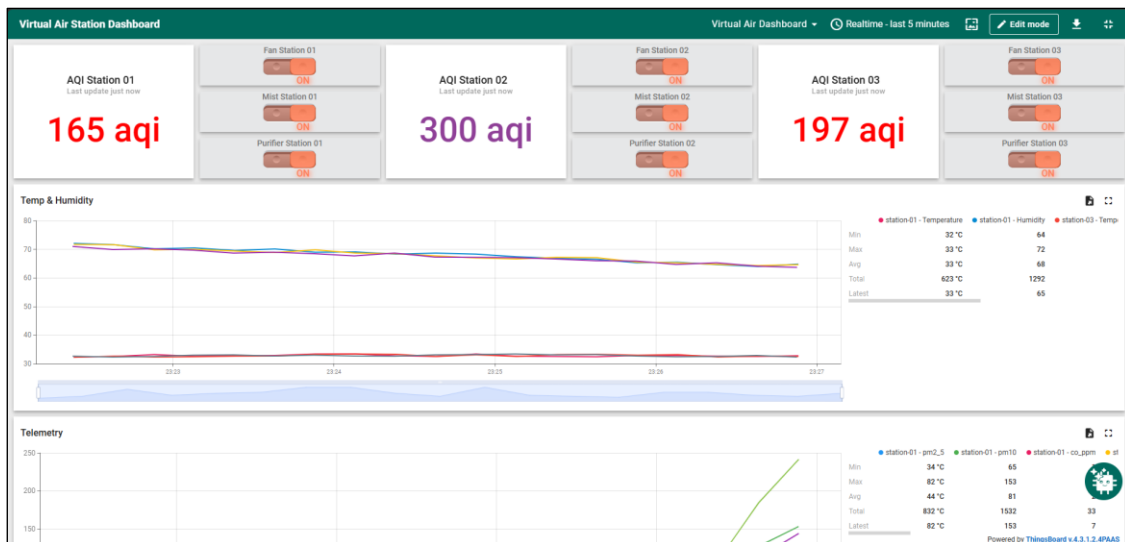
3.6. Lưu trữ InfluxDB



Module `influx.py` ghi dữ liệu vào InfluxDB v2 theo dạng thủ tục (gọi `init()` một lần, sau đó dùng các hàm `write_*`). Nhóm thiết kế ba measurement:

- **air_quality**: tag `city_id`, `station_id`; field `pm2_5`, `pm10`, `co_ppm`, `no2_ppb`, `o3_ppb`, `temperature`, `humidity` và `aqi`.
- **gateway_events**: tag `station_id`, `event_type`, `severity`, `dominant`; field `aqi`, `value`, `threshold`, `action_taken`.
- **actuator_status**: tag `station_id`; field `fan`, `purifier`, `mist`, `board`.

3.7. Tích hợp ThingsBoard (uplink + RPC + Dashboard + Cảnh báo)



Module `thingsboard.py` tích hợp ThingsBoard Cloud theo Gateway API và có thể bật/tắt qua biến `TB_ENABLED` (nếu `false` thì mọi hàm là no-op, hệ thống local vẫn chạy bình thường). Hai chiều dữ liệu:

- **Uplink:** hàm `send_telemetry()` publish telemetry nhiều trạm kèm AQI lên topic `v1/gateway/telemetry` theo định dạng `{station: [{ts, values}]}`; `connect_device()` báo cho ThingsBoard biết một thiết bị con đã online.
- **Downlink (RPC):** subscribe `v1/gateway/rpc`, nhận lệnh `setFan/setPurifier/setMist/setBoard`, chuyển thành command MQTT tới actuator qua `make_rpc_handler()`, rồi trả reply về ThingsBoard.

Để kịp thời cảnh báo chất lượng không khí, nhóm cấu hình thêm một luồng cảnh báo qua Alarm Thingsboard dựa trên dữ liệu thu thập được.

Created time	Originator	Type	Severity	Assignee	Status	Details
2026-06-24 23:22:43	station-03	Station 03 AQI	Critical	Thánh Đạt	Active Unacknowledged	---
2026-06-24 23:21:29	station-02	Station 02 AQI	Critical	Thánh Đạt	Active Unacknowledged	---
2026-06-24 23:13:37	station-01	Station 01 AQI	Critical	Thánh Đạt	Active Unacknowledged	---

3.8. REST API (FastAPI)

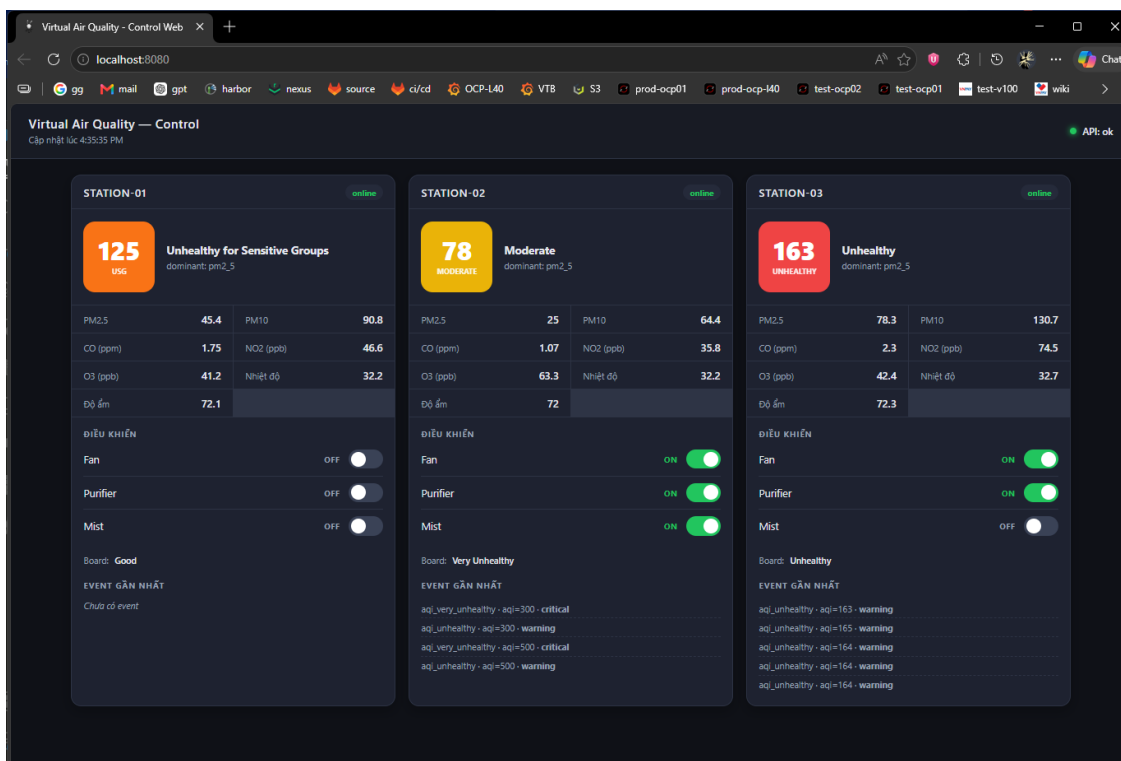
Service `air-api` dùng FastAPI, chạy một MQTT background client (khởi tạo trong lifespan) để subscribe `normalized/event/status` và giữ trạng thái mới nhất trong bộ nhớ (STATE và EVENTS deque tối đa 50 event/trạm). API cung cấp năm endpoint:

Method + Endpoint	Chức năng
GET /health	Kiểm tra trạng thái hệ thống + kết nối MQTT
GET /stations	Danh sách các trạm kèm AQI, dominant, category
GET /stations/{id}/state	Trạng thái hiện tại của một trạm
GET /stations/{id}/events	Danh sách event gần đây của trạm
POST /stations/{id}/command	Gửi lệnh điều khiển thủ công (publish MQTT)

Bảng 3.3. Các endpoint REST API

3.9. Control Web dashboard

Service `control-web` là một FastAPI đứng trước `air-api`, phục vụ trang dashboard HTML và proxy các lời gọi API. Giao diện hiển thị mỗi trạm là một card gồm: chỉ số AQI lớn có màu thay đổi theo cấp độ, nhãn category và dominant, lưới các thông số PM2.5/PM10/CO/NO₂/O₃/nhiệt độ/độ ẩm, các công tắc bật–tắt actuator (fan, purifier, mist) gửi command qua API, danh sách event gần nhất và một chỉ báo kết nối. Trang tự động làm mới theo chu kỳ `REFRESH_MS`.

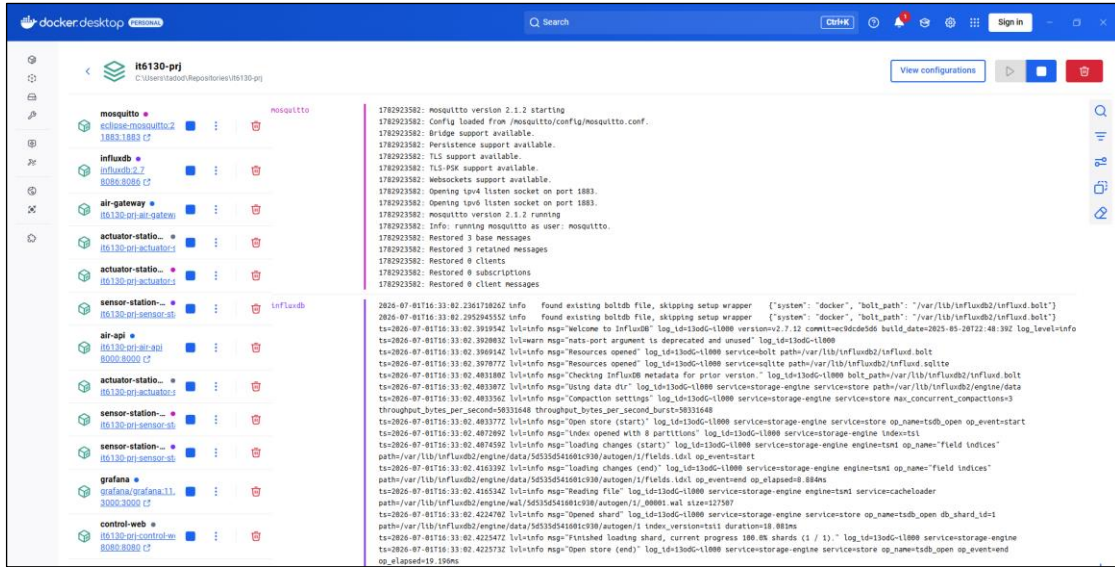


3.10. Grafana dashboard

Grafana được provisioning sẵn datasource InfluxDB (dùng ngôn ngữ truy vấn Flux) và một dashboard air_quality gồm sáu panel: AQI theo trạm; PM2.5 và PM10 theo thời gian; nồng độ CO/NO₂/O₃ theo trạm; bảng trạng thái actuator mới nhất; số event theo cấp severity theo thời gian; và bảng event gần nhất.



3.11. Docker Compose và triển khai



Toàn bộ 12 service được điều phối bằng một file docker-compose.yml: broker, InfluxDB, Grafana, gateway, ba sensor, ba actuator, REST API và control-web. Các service dùng chung mạng bridge air-net và gọi nhau bằng tên service. Compose sử dụng healthcheck (mosquitto, influxdb, air-api) kết hợp depends_on: condition: service_healthy để đảm bảo thứ tự khởi động; volume để lưu dữ liệu Mosquitto/InfluxDB/Grafana; và biến môi trường tập trung trong file .env. Việc triển khai chỉ gồm hai lệnh:

```
cp .env.example .env      # chỉnh sửa nếu cần
docker compose up -d --build
docker compose ps
```

CHƯƠNG 4. PHÂN CÔNG NHIỆM VỤ VÀ ĐÓNG GÓP CỦA CÁC THÀNH VIÊN

4.1. Nguyên tắc phân công

Nhóm chia hệ thống thành ba mảng công việc có khối lượng và độ khó tương đương, mỗi thành viên phụ trách trọn vẹn một mảng gồm nhiều service và chịu trách nhiệm từ thiết kế, cài đặt đến kiểm thử phần của mình. Việc phân chia bám theo ranh giới kỹ thuật tự nhiên của hệ thống (thiết bị ảo & giao tiếp – xử lý tại biên – cloud/API/triển khai) để giảm phụ thuộc chéo, đồng thời đảm bảo mỗi người đều chạm tới đủ các kỹ năng cốt lõi: mô phỏng dữ liệu, lập trình MQTT, xử lý – tính toán, tích hợp hệ thống và triển khai container. Ba mảng được cân đối để không thành viên nào có khối lượng nhẹ hơn đáng kể.

Thành viên (MSSV)	Mảng phụ trách	Các thành phần chính
Nguyễn Trường Chinh (20261101M)	Thiết bị ảo & Thiết kế giao tiếp MQTT	sensor.py, actuator.py, thiết kế topic + schema JSON, mosquitto.conf, Dockerfile sensor/actuator
Nguyễn Hà Minh (20261102M)	Edge Gateway & Xử lý dữ liệu	gateway.py (core), aqi.py, rules.py, influx.py, Dockerfile gateway
Đỗ Thành Đạt (20261103M)	Cloud, REST API, Dashboard & Triển khai	thingsboard.py + RPC handler, api/main.py, control-web, Grafana, docker-compose.yml, README

Bảng 4.1. Tổng quan phân công theo mảng

4.2. Nguyễn Trường Chinh (20261101M) – Thiết bị ảo & Thiết kế giao tiếp MQTT

Phụ trách toàn bộ tầng thiết bị đầu cuối và lớp giao tiếp MQTT — nền tảng để các mảng khác xử lý. Công việc chi tiết:

a) Thiết kế cây topic MQTT và định dạng message JSON

- Thiết kế cây topic phân cấp theo station_id gồm 5 nhánh (sensor/telemetry, actuator/command, actuator/status, gateway/normalized, gateway/event), đảm bảo mỗi trạm có namespace riêng và có thể thêm trạm mới chỉ bằng cách đổi station_id, không phải sửa mã.
- Thiết kế schema JSON thống nhất cho telemetry, command và status: quy ước các trường định danh device_id, station_id và timestamp theo ISO-8601 UTC; thống nhất tên trường nồng độ (pm2_5, pm10, co_ppm, no2_ppb, o3_ppb) dùng chung cho cả hệ thống.
- Chọn chính sách QoS 1 cho mọi bản tin và cờ retain cho status của actuator, để trạng thái mới nhất luôn sẵn có cho client kết nối sau.

b) Cài đặt Virtual Sensor (sensor.py)

- Đọc toàn bộ cấu hình từ biến môi trường (CITY_ID, STATION_ID, DEVICE_ID, MQTT_BROKER, MQTT_PORT, PUBLISH_INTERVAL) để cùng một image chạy được cho cả ba trạm.

- Xây dựng bảng baseline STATION_BASE với mức nền ô nhiễm khác nhau cho từng trạm, tạo sự khác biệt quan sát được trên dashboard.
- Cài đặt hàm diurnal_factor() mô phỏng hai đỉnh cao điểm giao thông (8h, 18h) bằng hai đường Gaussian.
- Cài đặt mô hình random walk có xu hướng qua hàm step_toward(): giá trị mới bám dần về mục tiêu (baseline $\times$ nhịp ngày) cộng nhiễu nhỏ, cho đường cong mượt thay vì số ngẫu nhiên rời rạc.
- Cài đặt cơ chế sinh tình huống ô nhiễm đột biến: sau ~40 vòng, kích hoạt một đợt spike kéo dài 4–8 vòng nhân PM lên 3,2 lần và đẩy CO vượt 9 ppm; mô phỏng O₃ ngược pha giao thông; sinh kèm nhiệt độ, độ ẩm theo hàm sin/cos.
- Cài đặt publish JSON telemetry theo chu kỳ PUBLISH_INTERVAL với QoS 1; xử lý callback on_connect/on_disconnect và in log gọn từng bản tin.
- Cài đặt cơ chế reconnect tự động (reconnect_delay_set 1–30s), vòng lặp thử kết nối tới khi broker sẵn sàng lúc khởi động, và tắt sạch container khi nhận SIGTERM/SIGINT.

c) Cài đặt Virtual Actuator (actuator.py)

- Subscribe topic command của trạm (QoS 1) và duy trì máy trạng thái STATE cho bốn thiết bị: fan (off/on/max), purifier (off/on), mist (off/on), board (Good/Moderate/Unhealthy/Very Unhealthy).
- Cài đặt hàm apply_command() phân tích command {target, action, reason}, cập nhật đúng trạng thái thiết bị (xử lý riêng board vì action là nhãn hiển thị), và bỏ qua command không hợp lệ.
- Cài đặt publish_status() phát trạng thái với retain=True; công bố trạng thái ban đầu ngay khi kết nối; reconnect tự động và tắt sạch bằng tín hiệu hệ thống.

d) Cấu hình broker và đóng gói

- Viết cấu hình Mosquitto (mosquitto.conf): listener 1883, allow_anonymous, bật persistence và logging.
- Viết Dockerfile cho sensor và actuator; khai báo requirements (paho-mqtt) và tham số môi trường trong docker-compose cho ba cặp sensor-actuator.

4.3. Nguyễn Hà Minh (20261102M) – Edge Gateway & Xử lý dữ liệu

Phụ trách bộ não xử lý tại biên: nhận, làm sạch, tính toán, ra quyết định và lưu trữ dữ liệu. Công việc chi tiết:

a) Lỗi điều phối Gateway (gateway.py)

- Cài đặt MQTT client của gateway: subscribe đồng thời hai wildcard city/+sensor/telemetry và city/+actuator/status (QoS 1), định tuyến bản tin theo hậu tố topic.
- Cài đặt hàm validate_normalize(): kiểm tra các trường bắt buộc (station_id, pm2_5, pm10) và loại bỏ bản ghi thiếu; ép kiểu float cho các trường số và kẹp giá trị âm về 0 để chống dữ liệu lỗi.

- Cài đặt hàm `handle_telemetry()` điều phối toàn bộ pipeline: tính AQI → cập nhật bộ nhớ LATEST → ghi InfluxDB → publish normalized → chạy rule engine → publish command và event → đẩy ThingsBoard, kèm log chi tiết từng bước.
- Cài đặt hàm `handle_status()` cập nhật trạng thái actuator vào LATEST và chuyển tiếp lên tầng lưu trữ/cloud.
- Thiết kế bộ nhớ trạng thái LATEST lưu bản đo, AQI và thời điểm cập nhật mới nhất của từng trạm, làm nguồn dữ liệu gián tiếp cho REST API.

b) Tính chỉ số AQI (aqi.py)

- Số hóa bảng breakpoint US EPA cho năm chất PM2.5, PM10, CO, NO₂, O₃ (mỗi mục gồm C_lo, C_hi, I_lo, I_hi).
- Cài đặt hàm `aqi_for()` nội suy tuyến tính AQI thành phần theo công thức EPA, kẹp ở mức trần khi nồng độ vượt bảng và trả None cho chất không hỗ trợ.
- Cài đặt hàm `compute_aqi()` lấy AQI tổng = max các thành phần và xác định dominant pollutant; cài các hàm `category()` và `board_label()` để phân cấp thành nhãn hiển thị.

c) Rule engine (rules.py)

- Thiết kế bảng ngưỡng THRESHOLDS tách rời khỏi logic để có thể chỉnh động từ Shared Attributes của ThingsBoard.
- Cài đặt bốn luật điều khiển theo cấp AQI: AQI>150 (bật purifier+fan, board Unhealthy, warning); AQI>200 (bật thêm mist, board Very Unhealthy, critical); CO>9 ppm (bật fan max, event co_high, critical); AQI≤50 (tắt toàn bộ thiết bị, board Good, info).
- Sinh song song danh sách command (điều khiển) và event (cảnh báo), gắn chuỗi `action_taken` vào mỗi event để phục vụ log và dashboard.

d) Lưu trữ InfluxDB (influx.py) và đóng gói

- Cài đặt module ghi InfluxDB v2 dạng thủ tục: `init()` kết nối theo URL/TOKEN/ORG/BUCKET từ env, dùng `write_api` chế độ SYNCHRONOUS, có bắt lỗi ghi.
- Thiết kế schema và cài ba hàm ghi: `write_air_quality` (measurement air_quality), `write_event` (gateway_events) và `write_actuator_status` (actuator_status) với tag/field đúng thiết kế.
- Viết Dockerfile cho gateway và khai báo phụ thuộc (paho-mqtt, influxdb-client).

4.4. Đồ Thành Đạt (20261103M) – Cloud, REST API, Dashboard & Triển khai

Phụ trách các tầng phía trên và ngoài edge: đồng bộ cloud, giao diện lập trình, trực quan hóa và điều phối triển khai toàn hệ thống. Công việc chi tiết:

a) Tích hợp ThingsBoard Cloud (thingsboard.py + RPC handler)

- Cài đặt uplink theo Gateway API: hàm send_telemetry() đóng gói telemetry nhiều trạm kèm AQI theo định dạng {station: [{ts, values}]} và publish lên v1/gateway/telemetry; hàm connect_device() báo thiết bị con online.
- Cài đặt downlink RPC: subscribe v1/gateway/rpc, phân tích {device, method, params}, gọi callback xử lý và publish reply trở lại ThingsBoard.
- Cài đặt cơ chế bật/tắt qua TB_ENABLED (no-op khi false) để hệ thống local chạy độc lập không cần tài khoản cloud.
- Cài đặt make_rpc_handler() trong gateway: ánh xạ setFan/setPurifier/setMist/setBoard thành command MQTT gửi tới actuator và trả kết quả.

b) REST API (api/main.py)

- Xây dựng ứng dụng FastAPI với MQTT background client khởi tạo theo lifespan, subscribe normalized/event/status và lưu STATE cùng EVENTS (deque tối đa 50 event/trạm).
- Cài đặt năm endpoint: GET /health, GET /stations, GET /stations/{id}/state, GET /stations/{id}/events và POST /stations/{id}/command; định nghĩa Pydantic model Command; xử lý lỗi 404/503 hợp lý.

c) Control Web dashboard (control-web)

- Xây dựng FastAPI proxy đứng trước air-api và phục vụ trang dashboard HTML (tiêm tham số REFRESH_MS, STATIONS vào template).
- Thiết kế giao diện realtime: mỗi trạm một card với chỉ số AQI đổi màu theo cấp, nhãn category/dominant, lưới thông số, công tắc bật–tắt actuator gửi command qua API, danh sách event và chỉ báo kết nối; tự làm mới theo chu kỳ.

d) Trực quan hóa và triển khai

- Provisioning Grafana: cấu hình datasource InfluxDB (Flux) và dashboard air_quality gồm sáu panel (AQI theo trạm, PM2.5/PM10, CO/NO₂/O₃, bảng actuator, số event theo severity, bảng event gần nhất).
- Viết docker-compose.yml điều phối 12 service: mạng bridge air-net, volume cho Mosquitto/InfluxDB/Grafana, healthcheck + depends_on: service_healthy để đảm bảo thứ tự khởi động, ánh xạ cổng và biến môi trường.
- Viết file .env.example tập trung cấu hình và README hướng dẫn kiến trúc, cách chạy, công truy cập và cấu trúc thư mục.

4.5. Bảng tổng hợp đóng góp

Thành viên	Số module chính	Đóng góp ước lượng	Vai trò
Nguyễn Trường Chinh	sensor.py, actuator.py, mosquitto.conf, 2 Dockerfile, thiết kế topic/JSON	≈ 33%	Thiết bị ảo & giao tiếp MQTT
Nguyễn Hà Minh	gateway.py, aqi.py, rules.py, influx.py, Dockerfile	≈ 33%	Edge gateway & xử lý dữ liệu
Đỗ Thành Đạt	thingsboard.py, api/main.py, control-web, Grafana, docker-compose, README	≈ 34%	Cloud, API, dashboard & triển khai

Bảng 4.2. Tổng hợp đóng góp của các thành viên

4.6. Công việc thực hiện chung

Ngoài phần phụ trách riêng, cả ba thành viên cùng tham gia: thống nhất kiến trúc và hợp đồng dữ liệu (topic, schema JSON) ở giai đoạn đầu; kiểm thử tích hợp end-to-end (sensor → gateway → actuator/DB/cloud/API); dò và sửa lỗi liên service; chuẩn bị kịch bản demo; và cùng biên soạn báo cáo cùng bộ slide trình bày.

CHƯƠNG 5. GIẢI ĐÁP CÁC CÂU HỎI BẮT BUỘC

Câu 1. Vì sao nên tính AQI tại gateway thay vì gửi dữ liệu thô lên cloud rồi mới tính?

Tính AQI ngay tại gateway (edge computing) mang lại nhiều lợi ích: (1) độ trễ thấp — gateway có thể phát hiện ô nhiễm và điều khiển actuator tức thời mà không chờ vòng lên—xuống cloud; (2) tính sẵn sàng — khi mất kết nối Internet, hệ thống vẫn tiếp tục cảnh báo và điều khiển cục bộ; (3) tiết kiệm băng thông — chỉ đẩy lên cloud dữ liệu đã chuẩn hóa kèm AQI thay vì mọi bản ghi thô; (4) giảm tải cho cloud vì phần tính toán đã phân tán về biên. Cloud khi đó chỉ đảm nhận giám sát tập trung và điều khiển từ xa (RPC).

Câu 2. Nhóm thiết kế topic và message format cho quan trắc không khí thế nào?

Nhóm dùng cây topic phân cấp theo station_id với năm nhánh: sensor/telemetry, actuator/command, actuator/status, gateway/normalized và gateway/event; mỗi trạm có namespace riêng nên thêm trạm mới chỉ cần đổi station_id. Message dùng JSON, thống nhất các trường định danh device_id, station_id và timestamp ISO-8601 UTC; tên trường nồng độ nhất quán trên toàn hệ thống. Chính sách QoS 1 cho mọi bản tin và retain cho status giúp giao nhận tin cậy và luôn có trạng thái mới nhất.

Câu 3. Cách tính AQI và xác định chất ô nhiễm dominant của nhóm?

Nhóm tính AQI theo bảng breakpoint US EPA bằng nội suy tuyến tính $I = (I_{hi} - I_{lo}) / (C_{hi} - C_{lo}) \times (C - C_{lo}) + I_{lo}$ cho từng chất (PM2.5, PM10, CO, NO₂, O₃). AQI tổng của trạm là giá trị lớn nhất trong các AQI thành phần; chất tạo ra giá trị lớn nhất chính là dominant pollutant. Nếu nồng độ vượt bảng thì kẹp ở mức trần. Cách này phản ánh đúng nguyên tắc: AQI đại diện cho chất ô nhiễm nguy hiểm nhất tại thời điểm đo.

Câu 4. Rule engine điều khiển lọc khí/quạt theo cấp AQI ra sao?

Rule engine so sánh AQI và nồng độ với các ngưỡng trong THRESHOLDS để sinh command và event. Bốn luật: AQI>150 bật purifier và fan, đặt board = Unhealthy (warning); AQI>200 bật thêm mist, board = Very Unhealthy (critical); CO>9 ppm bật fan mức max và sinh event co_high (critical, vì CO nguy hiểm trực tiếp); AQI≤50 tắt toàn bộ thiết bị và đặt board = Good (info). Các ngưỡng tách rời khỏi logic nên có thể chỉnh động từ cloud.

Câu 5. Gateway đóng vai trò ThingsBoard Gateway như thế nào?

Gateway hoạt động như một ThingsBoard Gateway đại diện cho nhiều thiết bị con: thay vì mỗi trạm tự kết nối cloud, gateway gộp telemetry của cả ba trạm và publish theo Gateway API lên topic v1/gateway/telemetry với định dạng {station: [{ts, values}]} (kèm AQI), đồng thời báo thiết bị online qua v1/gateway/connect. Ở chiều ngược lại, gateway subscribe v1/gateway/rpc để nhận lệnh điều khiển từ trung tâm, chuyển thành command MQTT nội

bộ tới actuator rồi trả reply. Nhờ vậy chỉ cần một điểm kết nối cloud cho toàn bộ mạng trạm.

Câu 6. So sánh dashboard Grafana (edge) và bản đồ AQI trên ThingsBoard (cloud)?

Tiêu chí	Grafana (edge)	ThingsBoard (cloud)
Vị trí	Chạy cục bộ cạnh gateway	Trên cloud, truy cập từ xa
Nguồn dữ liệu	Truy vấn trực tiếp InfluxDB (Flux)	Telemetry đẩy lên qua Gateway API
Thế mạnh	Phân tích chuỗi thời gian chi tiết, nhiều panel	Giám sát tập trung nhiều site, widget bản đồ, điều khiển RPC
Điều khiển	Chủ yếu để quan sát	Gửi lệnh RPC điều khiển thiết bị từ xa
Phụ thuộc mạng	Hoạt động cả khi mất Internet	Cần kết nối Internet

Bảng 5.1. So sánh Grafana (edge) và ThingsBoard (cloud)

Câu 7. Vì sao container không nên dùng localhost để gọi service khác?

Trong Docker, mỗi container có network namespace riêng nên localhost (127.0.0.1) chỉ trở về chính container đó, không phải máy host hay container khác. Nếu gateway gọi localhost:1883 nó sẽ tìm broker bên trong chính nó và thất bại. Vì vậy các service phải gọi nhau bằng tên service (ví dụ mosquito, influxdb, air-api) — Docker Compose cung cấp DNS nội bộ trên mạng air-net để phân giải tên service thành IP container tương ứng.

Câu 8. Muốn mở rộng thành mạng quan trắc toàn thành phố cần thay đổi gì?

Nhờ thiết kế theo station_id, việc thêm trạm chỉ cần khai báo service sensor/actuator mới với station_id mới — không phải sửa logic gateway (vốn dùng wildcard). Để mở rộng quy mô toàn thành phố cần bổ sung: nhiều gateway theo khu vực (mỗi gateway phụ trách một cụm trạm) để phân tán tải; broker MQTT dạng cluster hoặc dùng bridge; InfluxDB với chính sách lưu trữ/retention và downsampling cho dữ liệu lớn; cơ chế bảo mật (xác thực MQTT, TLS, phân quyền) thay cho allow_anonymous; và widget bản đồ (map) trên ThingsBoard để hiển thị AQI theo tọa độ các trạm.

CHƯƠNG 6. KẾT QUẢ VÀ ĐÁNH GIÁ

6.1. Kết quả đạt được

Hệ thống chạy được toàn bộ 12 container bằng một lệnh docker compose up. Nhóm đã hoàn thành và kiểm thử các chức năng:

- Mô phỏng ba trạm quan trắc, telemetry biến thiên theo nhịp ngày và có tình huống đột biến.
- Gateway chuẩn hóa dữ liệu, tính AQI và xác định dominant pollutant chính xác.
- Rule engine tự động điều khiển fan/purifier/mist/board theo cấp AQI; actuator phản hồi status.
- Lưu telemetry, AQI, event và trạng thái vào InfluxDB; Grafana hiển thị đủ sáu panel.
- Tích hợp ThingsBoard: đẩy telemetry kèm AQI và điều khiển từ xa qua RPC.
- REST API và Web dashboard truy vấn trạng thái, gửi lệnh điều khiển thủ công thành công.

6.2. Đối chiếu tiêu chí chấm điểm

Đối chiếu với thang điểm mục 4.16 của đề bài, hệ thống đáp ứng đầy đủ các hạng mục bắt buộc:

Nội dung	Điểm	Đáp ứng
Kiến trúc Virtual Air Quality Gateway hợp lý	1.0	Có
Thiết kế MQTT topic & message format rõ ràng, mở rộng được	1.0	Có
Virtual sensor mô phỏng bụi/khí có xu hướng và tình huống ô nhiễm	1.0	Có
Virtual actuator nhận command và publish status	0.8	Có
Gateway tính AQI chính xác, normalize, lưu trạng thái	0.8	Có
Rule engine điều khiển lọc khí theo cấp AQI tự động	1.0	Có
Tích hợp ThingsBoard: telemetry gateway + RPC điều khiển từ xa	1.2	Có
Ghi telemetry/AQI/event/status vào InfluxDB	0.8	Có
Grafana & FastAPI dashboard thể hiện rõ AQI, event, trạng thái	0.8	Có
Docker Compose, Dockerfile, network, volume, env đúng	1.0	Có
README, báo cáo, demo và giải thích hệ thống	0.7 + 0.5	Có

Bảng 6.1. Đối chiếu với tiêu chí chấm điểm (tổng 10.0 điểm)

6.3. Một số yêu cầu nâng cao đã thực hiện

Ngoài yêu cầu bắt buộc, nhóm thực hiện thêm một số yêu cầu nâng cao (mục 4.17): tính AQI đầy đủ cho nhiều chất và xác định dominant; ngưỡng rule có thể chỉnh động qua Shared Attributes; health check cho các service (mosquitto, influxdb, air-api) trong Docker Compose; cơ chế alarm/event theo cấp severity; và tự động reconnect MQTT cho mọi client.

6.4. Hạn chế và hướng phát triển

Hệ thống hiện dùng broker mở (allow_anonymous) và chưa có bảo mật TLS/xác thực, phù hợp môi trường mô phỏng nhưng cần bổ sung khi triển khai thật. AQI được tính trên giá trị tức thời thay vì trung bình trượt theo cửa sổ thời gian như chuẩn EPA. Hướng phát triển gồm: bổ sung bảo mật, tính AQI theo trung bình trượt, thêm mô-đun dự báo AQI ngắn hạn, viết unit test tự động, và mở rộng thành mạng nhiều gateway theo khu vực kèm widget bản đồ.

KẾT LUẬN

Nhóm đã xây dựng thành công hệ thống Virtual Smart Air Quality Gateway hoàn chỉnh theo Đề tài 4, ảo hóa toàn bộ bằng Docker Compose. Hệ thống tái hiện đầy đủ một mạng quan trắc không khí thực tế: nhiều trạm cảm biến ảo gửi telemetry qua MQTT, một edge gateway đứng ở biên chuẩn hóa dữ liệu, tính AQI, chạy rule engine điều khiển thiết bị chấp hành, lưu trữ vào InfluxDB, trực quan hóa trên Grafana, đồng bộ hai chiều với ThingsBoard Cloud và cung cấp REST API cùng Web dashboard.

Đối chiếu với đề bài, hệ thống đáp ứng đầy đủ các yêu cầu bắt buộc và thực hiện thêm một số yêu cầu nâng cao. Điểm mấu chốt của thiết kế là mô hình edge computing giúp xử lý và điều khiển kịp thời tại chỗ, cùng cách tổ chức topic và message theo station_id giúp hệ thống dễ dàng mở rộng.

Qua đồ án, các thành viên đã rèn luyện và củng cố nhiều kiến thức, kỹ năng: lập trình mạng với MQTT, mô phỏng dữ liệu IoT có ý nghĩa, tính toán chỉ số môi trường AQI, xây dựng rule engine, làm việc với cơ sở dữ liệu time-series, tích hợp nền tảng IoT cloud, xây dựng REST API và dashboard, cùng kỹ năng đóng gói – điều phối microservice bằng container. Ngoài kiến thức kỹ thuật, nhóm cũng nâng cao kỹ năng làm việc nhóm, phân chia công việc, tích hợp hệ thống, tổng hợp tài liệu và trình bày báo cáo.

PHỤ LỤC – TRÍCH DẪN MÃ NGUỒN

Link toàn bộ mã nguồn đề tài: TadodMaster/it6130-prj: Virtual Smart Air Quality Gateway

Phụ lục trích một số đoạn mã tiêu biểu của từng thành viên (đã lược bớt để gọn). Mã nguồn đầy đủ được tổ chức trong thư mục services/ và infra/ của dự án.

A. Nguyễn Trường Chinh – Sensor & Actuator

A.1. Mô phỏng dữ liệu có xu hướng và đột biến (sensor.py)

```
def diurnal_factor() -> float:
    """Hệ số theo giờ: cao điểm giao thông sáng (~8h) và chiều (~18h)."""
    hour = datetime.now(timezone.utc).hour
    morning = math.exp(-((hour - 8) ** 2) / 5.0)
    evening = math.exp(-((hour - 18) ** 2) / 5.0)
    return 1.0 + 0.7 * (morning + evening)

def step_toward(current, target, jitter):
    """Di chuyển dần current về target + nhiễu -> đường cong mượt có xu hướng."""
    nxt = current + (target - current) * 0.3 + random.uniform(-jitter, jitter)
    return max(0.0, nxt)

def simulate() -> dict:
    global tick, spike_remaining
    tick += 1
    f = diurnal_factor()
    if spike_remaining == 0 and tick % 40 == 0:      # ~40 vòng sinh 1 đợt spike
        spike_remaining = random.randint(4, 8)
    spike = spike_remaining > 0
    if spike:
        spike_remaining -= 1
    pm_mult = 3.2 if spike else 1.0                  # bụi mịn tăng vọt khi đột biến
    co_mult = 9.0 / BASE["co"] if spike else 1.0     # đẩy CO vượt ngưỡng 9 ppm
    state["pm2_5"] = step_toward(state["pm2_5"], BASE["pm2_5"] * f * pm_mult, 4)
    ...
```

A.2. Máy trạng thái actuator (actuator.py)

```
def apply_command(cmd: dict):
    """Cập nhật STATE theo command JSON: {target, action, reason}."""
    global last_reason
    target = cmd.get("target")
    action = cmd.get("action")
    last_reason = cmd.get("reason", last_reason)
    if target == "board":
        STATE["board"] = action          # action là nhãn hiển thị
    elif target in ("fan", "purifier", "mist"):
        STATE[target] = action           # on / off / max
    else:
        print(f"[{DEVICE_ID}] command không hợp lệ: {cmd}", flush=True)
        return False
    return True
```

B. Nguyễn Hà Minh – Gateway, AQI & Rule engine

B.1. Chuẩn hóa dữ liệu (gateway.py)

```
def validate_normalize(m: dict):
    """Kiểm tra trường bắt buộc, ép kiểu số, kẹp giá trị âm về 0."""
    for f in REQUIRED_FIELDS: # ("station_id", "pm2_5", "pm10")
        if f not in m:
            print(f"[gateway] bỏ qua bản ghi thiếu '{f}': {m}", flush=True)
            return None
    out = dict(m)
    for f in ("pm2_5", "pm10", "co_ppm", "no2_ppb", "o3_ppb", "temperature", "humidity"):
        try:
            out[f] = max(0.0, float(m.get(f, 0)))
        except (TypeError, ValueError):
            out[f] = 0.0
    return out
```

B.2. Nội suy AQI và xác định dominant (aqi.py)

```
def aqi_for(pollutant, conc):
    """AQI thành phần cho một chất tại nồng độ conc (nội suy tuyến tính)."""
    table = BREAKPOINTS.get(pollutant)
    if table is None or conc is None:
        return None
    conc = max(0.0, float(conc))
    for c_lo, c_hi, i_lo, i_hi in table:
        if c_lo <= conc <= c_hi:
            return (i_hi - i_lo) / (c_hi - c_lo) * (conc - c_lo) + i_lo
    return float(table[-1][3]) # vượt bảng -> kẹp mức trần

def compute_aqi(measurement):
    components = {}
    for p in ("pm2_5", "pm10", "co_ppm", "no2_ppb", "o3_ppb"):
        sub = aqi_for(p, measurement.get(p))
        if sub is not None:
            components[p] = round(sub)
    dominant = max(components, key=components.get) # AQI tổng = max thành phần
    return {"aqi": components[dominant], "dominant": dominant, ...}
```

B.3. Luật điều khiển theo cấp AQI (rules.py)

```
# --- Luật 1: AQI > 150 (Unhealthy) ---
if aqi > THRESHOLDS["aqi_unhealthy"]:
    cmd("purifier", "on", "aqi_unhealthy")
    cmd("fan", "on", "aqi_unhealthy")
    cmd("board", "Unhealthy", "aqi_unhealthy")
    events.append({"event_type": "aqi_unhealthy", "severity": "warning", ...})

# --- Luật 3: CO > 9 ppm (nguy hiểm trực tiếp) ---
if co > THRESHOLDS["co_high_ppm"]:
    cmd("fan", "max", "co_high")
    events.append({"event_type": "co_high", "severity": "critical", ...})
```

C. Đỗ Thành Đạt – Cloud, API & Triển khai

C.1. Uplink telemetry theo ThingsBoard Gateway API (thingsboard.py)

```
def send_telemetry(station_id, values, ts_ms):
```

```

"""Đẩy telemetry kèm AQI lên ThingsBoard theo Gateway API."""
if not ENABLED or not _client:
    return
payload = {"station_id": [{"ts": ts_ms, "values": values}]}
_client.publish("v1/gateway/telemetry", json.dumps(payload), qos=1)

```

C.2. Endpoint điều khiển thủ công (api/main.py)

```

@app.post("/stations/{station_id}/command")
def send_command(station_id: str, cmd: Command):
    if not (mqtt_client and mqtt_client.is_connected()):
        raise HTTPException(503, "MQTT chưa kết nối")
    payload = {"station_id": station_id, "target": cmd.target,
              "action": cmd.action, "reason": cmd.reason, "timestamp": now_iso()}
    mqtt_client.publish(f"city/{station_id}/actuator/command",
                       json.dumps(payload), qos=1)
    return {"published": True, "payload": payload}

```

C.3. Điều phối service bằng Docker Compose (docker-compose.yml)

```

air-gateway:
  build: ./services/gateway
  environment:
    MQTT_BROKER: ${MQTT_BROKER}          # gọi broker bằng TÊN service, không localhost
    INFLUXDB_URL: ${INFLUXDB_URL}
    TB_ENABLED: ${TB_ENABLED}
  depends_on:
    mosquitto: { condition: service_healthy }
    influxdb: { condition: service_healthy }
  networks: [air-net]
  restart: unless-stopped

```